

A Hierarchical ReAct Multi-Agent Pipeline for Residential Appliance Load Scheduling Under Dynamic Electricity Pricing

University of Oulu
Information Processing Science
Master's Thesis
Hoan Le
2026

Abstract

Households on dynamic electricity contracts can lower their bills by running flexible appliances in low-price hours, but coordinating several appliances under a volatile price curve and a household peak load limit is difficult without automation. This thesis follows Design Science Research to design and evaluate a hierarchical ReAct multi-agent system, built in LangGraph, that gives a household a daily appliance recommendation. An Orchestrator, an Appliance Profiler, a Price-Aware Scheduler and a Conflict Resolver work from the day's appliance plan, the base load, the peak limit and the prices, and a verification step returns flawed schedules for revision. The agents are given similar past days whose optimal schedules are known as worked examples, and an optimiser tool that they call, check and explain. The study asks how much the recommendations save, how often they are valid and what they cost to produce compared with a language model answering without tools (SQ1), and under which price patterns, household loads and peak limits the recommendations save more than, or as much as, existing home energy management tools available to households: cheapest-hours automations and the open-source optimiser EMHASS (SQ2).

The evaluation combines appliance cycles from the REFIT dataset with Finnish Nord Pool prices in 48 exploratory scenarios and tests pre-registered hypotheses on 48 held-out scenarios with fifteen-minute prices. The primary outcome is the saving against starting every appliance when it is ready. An earlier confirmatory run showed that tool use is essential: with GPT-4o-mini the pipeline saved 22 % of the cost on held-out days, whereas the same model without tools saved 6 %. Cheapest-hours automations, which plan each appliance on its own, exceed the household peak limit on about half of the days. The results for retrieval and the optimiser tool on the held-out set are in progress.

Keywords

multi-agent system, ReAct, LangGraph, retrieval-augmented generation, home energy management, appliance scheduling, dynamic electricity pricing, Design Science Research

Supervisor

Yueqiang Xu, PhD, Postdoctoral Researcher

Foreword

Work in progress.

Abbreviations

API	Application Programming Interface
DSR	Design Science Research
HEMS	Home Energy Management System
LLM	Large Language Model
MAS	Multi-Agent System
MCP	Model Context Protocol
MILP	Mixed Integer Linear Programming
Nord Pool FI	Nord Pool Finland Bidding Zone
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REFIT	Personalised Retrofit Decision Support Tools for UK Homes using Smart Home Technology (the research project that produced the dataset)
SLM	Small Language Model
SLR	Systematic Literature Review

Contents

Abstract **1**

Foreword **2**

Abbreviations **3**

1 Introduction **7**

1.1 Background and motivation 7

1.2 Research problem 8

1.3 Design Science Research approach 9

1.4 Structure of the thesis 9

2 Literature Review **10**

2.1 Home energy management systems 10

2.2 LLM-based agents for energy management 11

2.3 The ReAct framework 12

2.4 Multi-agent LLM systems 13

2.5 Retrieval and solver tools for LLM agents 14

2.6 Agentic AI trends in 2025–2026 14

2.7 Finnish energy market context 15

2.8 Literature synthesis and research gap 15

3 Research Methodology **17**

3.1 Design Science Research methodology 17

3.2 Theoretical foundations 18

3.3 Evaluation strategy 19

3.3.1 Data collection 21
3.3.2 Data analysis 22

4 System Design and Implementation **25**
4.1 Problem definition and objectives 25
4.2 Architecture overview 25
4.3 Agent specifications 27
4.4 Retrieval of solved days and the optimiser tool 29
4.5 MILP solver 30
4.6 Technology stack 31

5 Evaluation and Results **33**
5.1 Dataset and scenario design 33
5.1.1 Appliance data 33
5.1.2 Electricity price data 34
5.1.3 Scenario construction 35
5.2 Value of 15-minute price information 37
5.3 Demonstration results 38
5.3.1 Design iterations 39
5.4 Quantitative results 40
5.5 Statistical analysis 40
5.6 Comparison with baselines and real tools 41
5.7 Confirmatory evaluation 43
5.8 Per-appliance breakdown 44

6 Discussion **45**
6.1 Interpretation of research questions 45
6.2 Limitations 45
6.3 Future research 46

7 Conclusions **47**

References **48**

1. Introduction

Smart meters, dynamic electricity contracts and a growing share of renewable generation have changed what a household pays for electricity from hour to hour. A household on a spot-price contract can lower its bill by running flexible appliances, such as the washing machine or the dishwasher, in the day's low-price hours. Without automation this is hard when several appliance cycles have to fit a price curve that changes every day and the household must also stay under a limit on its total power.

Finland is a clear case. Since October 2025, Finnish consumers on spot-price contracts pay prices set for every 15 minutes on the Nordic spot market (Nord Pool FI bidding zone), and prices within a day vary widely. From 2023 to 2026, the most expensive hour of a day cost a median of 3.1 times the cheapest on days without near-zero prices, and one day in five had at least one hour with a negative price (computed from Nord Pool FI prices; Pakastin Oy, 2026). Such extreme prices measurably change household consumption and affect the distribution network (Haakana et al., 2025). Language models have recently been applied to the same problem: in the first Home Energy Management Systems (HEMS) built on large language models, agents schedule household appliances, a task that earlier needed hand-written rules or an optimization solver (El Makroum et al., 2025; Jonah et al., 2026). These first systems have documented limitations, which this thesis takes up.

1.1 Background and motivation

HEMS have mostly been built on mathematical optimization, rule-based schedulers, or machine learning models trained on past consumption (Beaudin & Zareipour, 2015). These approaches work well for a precisely specified problem, but they take expert effort to configure, and a rule or a trained model does not carry over to price conditions it was not designed for.

Large Language Models (LLMs) can be used as general reasoning engines, which gives HEMS design another option. Yao et al. (2023) introduce ReAct, a Thought-Action-Observation loop in which the model alternates between reasoning and tool calls, and show that it solves multi-step tasks that need information looked up along the way. For household scheduling, a ReAct agent can read the price forecast, decide which appliances can be shifted, find the cheapest time window for each and check the result against the household's constraints, and every step of that reasoning is recorded.

A secondary motivation concerns sustainable AI deployment. The energy needed per inference differs by orders of magnitude between models and tasks (Luccioni et al., 2024), and serving a small language model (SLM) of about 7 billion parameters has been estimated to be 10 to 30 times cheaper in latency, energy and computation than serving a model of 70 to 175 billion parameters (Belcak et al., 2025). Whether such smaller

models are good enough for a given agentic task is an empirical question. Since the Low Carbon Alert trainee project context prioritizes energy efficiency in the AI systems used to schedule household loads, the inference cost of each design, measured in tokens and response time, is reported as an outcome alongside its saving. A design that reaches the same saving with fewer model calls is cheaper to run and easier to move to a small local model later.

1.2 Research problem

In the hierarchical multi-agent system of El Makroum et al. (2025), only one of three evaluated models coordinated all appliances, and the evaluation covers a single day. Reviewing that work, Jonah et al. (2026) list multi-appliance coordination failures, scheduling on price alone, a single-day evaluation horizon and untested constraint safety as open problems. Their own single-agent system is evaluated for one household over a few weeks, and they recommend that every schedule be validated independently before it is executed. Neither study varies the price conditions. It is also unclear what a language model agent adds over the tools households can already use, such as cheapest-hours automations and open-source optimizers, and whether an agent can match an optimizer when several appliances have to be staggered. Two techniques used in other LLM systems could help but have not been tried for household scheduling: giving the model retrieved examples of solved problems (retrieval-augmented generation, RAG), and giving it a solver as a tool.

The artifact studied in this thesis gives each household a daily recommendation. Every day it receives the household's appliance plan for that day, namely which appliance cycles are to run and when each is ready and must be finished, together with the household's base load, its peak load limit and the day's electricity prices. It returns a start time for each appliance and a short explanation. To address the limitations above, the study is guided by one main research question and two sub-questions:

> **RQ:** How can a hierarchical ReAct multi-agent system be designed to give households reliable, cost-saving daily appliance recommendations under dynamic electricity prices?
>
> **SQ1 (value and cost):** How much do the pipeline's recommendations save, how often are they valid, and what do they cost to produce, compared with a language model that answers without tools?
>
> **SQ2 (conditions):** Under which price patterns, household loads and peak limits do the recommendations save more than, or as much as, existing home energy management tools available to households, namely cheapest-hours automations and the EMHASS optimizer?

The main RQ states the design aim: recommendations that are reliable, meaning that they respect every appliance window and the household peak limit, and that save money

against starting every appliance when it is ready. The pipeline studied is the final design: four ReAct agents that are shown how similar past days were solved best and that hand the search to an optimiser tool, then check the answer and explain it. SQ1 concerns what its recommendations are worth and what they cost. It tests the premise of the ReAct approach, that reasoning grounded in tool observations does better than unaided generation, by comparing the pipeline with the same language model answering directly, and it checks the response time and API cost of each recommendation against the design objectives. SQ2 concerns where the system is worth using. Its ”existing home energy management tools” are real, freely available software that a household can install without programming: the cheapest-hours automations shared for Home Assistant and Shelly smart relays, which switch a device on in the day’s cheapest hours, and the open-source optimizer EMHASS, which computes a schedule for the whole household. SQ2 compares the recommendations with these tools and asks how the comparison depends on the day’s prices, on how many appliances the household runs, and on how tight its peak limit is.

1.3 Design Science Research approach

This study follows the Design Science Research (DSR) methodology as defined by Peffers et al. (2007). The study builds a software artifact, the hierarchical ReAct multi-agent system, and evaluates it over several iterations, which is the kind of work DSR is meant for. As Hevner et al. (2004) explain, DSR produces knowledge about a design problem and its solution by building and evaluating an artifact. The artifact is the main research contribution. A Systematic Literature Review (SLR) forms DSR Phase 1: it confirms the research gap and grounds the architectural decisions in prior work.

1.4 Structure of the thesis

This thesis is organized into seven chapters. Chapter 2 presents the literature review covering HEMS concepts, LLM-based agents for energy management, the ReAct framework, multi-agent LLM systems, retrieval and solver tools for agents, current agentic AI trends, and the Finnish energy market context. Chapter 3 describes the Design Science Research methodology and evaluation strategy. Chapter 4 presents the system design and implementation. Chapter 5 presents the results on an exploratory and a held-out benchmark of 96 scenarios each. Chapter 6 discusses findings, limitations, and future work. Chapter 7 concludes the thesis.

2. Literature Review

This chapter reviews the background and prior empirical work on which the design and evaluation of the multi-agent system rest. It covers eight topics and ends with the research gap that the research questions address.

2.1 Home energy management systems

A Home Energy Management System (HEMS) is a software or hardware system that monitors, controls, and optimizes residential electricity consumption to reduce cost, support grid stability, and improve energy efficiency (Beaudin & Zareipour, 2015). The core scheduling problem is the assignment of flexible appliance loads to time windows where electricity prices are lowest, while ensuring that total household load at any moment does not exceed safe operational limits. A load is flexible when its start time can be shifted without violating user preferences or physical constraints.

HEMS scheduling approaches fall into three broad groups. Rule-based systems apply expert-defined heuristics, for example starting an appliance when the price falls below a threshold or in the cheapest hours of a window. They are simple to run and easy to follow, because each decision follows from one stated rule, but each rule covers only the situation it was written for. A rule that picks the cheapest hours for one device does not see the other devices, so it cannot keep the household under a limit on its total power, and a new requirement means writing and testing a new rule.

Mathematical optimization methods such as Mixed Integer Linear Programming (MILP) compute the best schedule for a precisely specified problem. Sou et al. (2011) formulate appliance scheduling with binary start variables, uninterrupted cycles and time windows, and Moradzadeh and Tomsovic (2013) formulate residential energy management as a two-stage optimization that respects the operational constraints of the distribution network. Such methods are optimal for the model they solve, but every constraint must first be expressed in mathematical form, so a household's preferences have to be translated into variables and constraints by someone who can do so, and a new kind of constraint needs a new formulation. Their output is a schedule, not a reason for it.

Learning-based methods acquire a scheduling policy from data, as in the deep reinforcement learning approach of Mocanu et al. (2019), which is trained on measured building data and then optimizes schedules on-line. They adapt to the household they are trained on, but a change in the task, such as a new appliance or a new tariff, generally requires new training, and the learned policy is hard to inspect.

Optimization and learning-based controllers have been described as black boxes that are hard for users to understand and trust, which has led to work on explainable HEMS,

for example by distilling a reinforcement-learning policy into a decision tree that can be explained to end-users (Gokhale et al., 2024). Zharova et al. (2024) argue that users need to understand why and how a recommendation system reaches its recommendations before they trust and accept them, and add a dedicated explainability agent to a multi-agent system that recommends when to run household appliances. Explanation in terms a non-expert understands, and adaptation to new constraint types without a new rule, formulation or training run, are therefore the two properties that a language model agent could add to these approaches.

Rule-based and optimization approaches are both available to households today as open-source tools. Among Home Assistant users in Finland and the other Nordic countries, a widely shared approach is to automate devices with community blueprints that switch a device on during the cheapest hours of a user-defined time window, based on Nord Pool day-ahead prices (Kert, n.d.). Related tools choose the block of consecutive hours with the lowest average price before a set end time, such as the Nordpool Planner integration for Home Assistant (dala318, n.d.), or switch Shelly smart relays on in the cheapest hours of a period, as the shelly-porssisahko script widely used in Finland does (jisotalo, n.d.). Such rules handle each device separately and do not consider the household's total load. EMHASS (Energy Management for Home Assistant) is an open-source optimizer that schedules deferrable loads, batteries and photovoltaic production against price forecasts with linear and mixed-integer programming (EMHASS contributors, 2026). It supports a time window for each load, a limit on the power drawn from the grid, and loads defined by a fixed power profile. Both kinds of tool require the household to describe each appliance in structured form, namely its power, run time and time window. OpenHEMS, a Home Assistant add-on, schedules heavy appliances under a maximum power but targets fixed off-peak tariffs rather than spot prices (OpenHEMS contributors, 2026). This thesis uses the cheapest-hours rules and EMHASS as reference points for the agents (Section 3.3).

2.2 LLM-based agents for energy management

Studies applying LLMs to energy management have appeared since 2023, and two recent ones are close to this thesis. El Makroum et al. (2025) present a hierarchical agentic architecture for home energy management in which an orchestrator agent delegates to three specialist agents that follow the ReAct pattern. The household has three flexible loads: a washing machine (2.0 kW for 120 minutes), a dishwasher (1.8 kW for 90 minutes) and an electric vehicle charger (7.4 kW for 360 minutes). The agents receive Austrian day-ahead prices from ENTSO-E for one day, 14 October 2025, and are run with three open-weight models, Llama 3.3 70B, Qwen 3 32B and GPT-OSS 120B. The evaluation consists of 75 runs: 15 single-appliance runs, 15 multi-appliance runs and 45 runs in which the user asks analytical questions about the schedule. Each schedule is compared with the optimum of a MILP model rather than with an unscheduled baseline, so the study reports how often the agents reach the optimal cost, not how much they save. All three models scheduled single appliances optimally, but only Llama 3.3 70B coordinated all appliances in every

multi-appliance scenario, and the analytical questions were answered unreliably unless the agents were given explicit guidance. The authors describe the study as a proof of concept: one household, one day and one price series, without thermal loads such as heat pumps, electric heating and hot water storage. They name pilot deployments and evaluation across electricity markets and household types as future work. Of the published systems, this design is the closest to the pipeline of this thesis.

Jonah et al. (2026) take the opposite design choice: a single tool-calling ReAct agent that turns a household's request into appliance schedules, using half-hourly dynamic prices from the United Kingdom (Octopus Agile), weather and photovoltaic forecasts and household demand data. They test five language models, among them Claude Sonnet 4.6, GPT-4o-mini and Qwen 3. Their agents capture 96.7 to 98.0 % of the savings available between an off-peak timer and a MILP optimum and do better than rule-based policies. Their failure analysis is the more important result for this thesis. The models sometimes fabricated schedules or failed to commit one, and in some cases a model explained a deadline correctly and still committed a schedule that violated it. Actions parsed from text were less reliable than native function calls. Building on El Makroum et al., they list multi-appliance coordination failures, scheduling on price alone, a single-day evaluation horizon and untested constraint safety as open problems, and they recommend that an independent deterministic validator check every schedule against duration, deadline and power constraints before it is executed. Their evaluation covers one household over twelve tariff days and a one-week deployment. Section 2.8 sets out how these open problems shape the design and evaluation of this thesis.

2.3 The ReAct framework

In the ReAct (Reasoning and Acting) framework of Yao et al. (2023), an LLM agent interleaves reasoning with actions instead of first planning everything and then acting. Each step has three parts: a Thought, which reasons about the current state and the next action; an Action, which is a structured call to an external tool or API; and an Observation, which is the result the tool returns. The loop repeats until the agent reaches a final answer.

Yao et al. (2023) evaluate ReAct on four benchmarks of two kinds. Two are knowledge-intensive reasoning tasks, in which the agent can query a Wikipedia API: HotpotQA, multi-hop question answering, and FEVER, fact verification. Two are interactive decision-making tasks: ALFWorld, a text-based game in which the agent carries out household chores in a simulated home, and WebShop, in which it shops on a simulated website to match a customer's instructions. Their baselines are built by removing parts of the ReAct trajectories: standard prompting without reasoning or actions, chain-of-thought prompting with reasoning but no actions, and acting alone without reasoning, so that each comparison isolates the contribution of reasoning or of acting. With PaLM-540B, ReAct reached 27.4 exact match on HotpotQA, slightly below chain-of-thought (29.4), and 60.9 % accuracy on FEVER, above it (56.3 %), while the best results on both came from combining the two. On the interactive tasks the advantage was clear: on ALFWorld the best ReAct trial reached a 71 % success rate against 45 %

for acting alone, and on WebShop 40.0 % against 30.1 %. The gains were largest where the agent has to act on information it looks up during a multi-step task. Household scheduling is such a task: the agent looks up prices, loads and costs while it builds the schedule. The trace also makes the result explainable, because it records the reasoning behind every tool call, so it shows why an appliance was given a particular start time.

2.4 Multi-agent LLM systems

Multi-agent LLM systems have been proposed as a solution to the context window and coordination limitations of single-agent designs. Han et al. (2024) discuss the challenges and open problems of LLM multi-agent systems, among them task allocation, robust reasoning through iterative debate, the management of complex and layered context information, and memory management. Role specialization, in which each agent is responsible for one well-defined subtask and has access only to the tools that subtask needs, is one response to these challenges and the design principle followed in this thesis.

Hong et al. (2024) demonstrate role-based multi-agent orchestration for software development with MetaGPT, in which agents act as product manager, architect, project manager, engineer and quality-assurance engineer. Encoding standard operating procedures and requiring structured intermediate outputs, exchanged through a shared message pool, reduces the inconsistencies that arise when language model calls are simply chained, and MetaGPT reported state-of-the-art results on code-generation benchmarks at the time. Structured hand-offs between specialists address the same kind of problem as the multi-appliance coordination failures reported for LLM-based HEMS (El Makroum et al., 2025).

The move from one agent to a hierarchy of agents is motivated by three problems of a single agent that handles a whole task. First, its context grows with every tool call and observation, and the model has to keep track of all of them, while a specialist sees only the information of its own step. Second, a single agent has access to every tool, so it can call any of them at any point, while a specialist is limited to the few tools its subtask needs, which narrows the choices it can get wrong. Third, the intermediate results of a single agent stay inside its own reasoning, while a hierarchy passes them between agents as structured outputs, which can be checked before the next step (Hong et al., 2024). In a hierarchical design an orchestrator splits the task, hands each part to a specialist and combines the results; the survey of Guo et al. (2024) describes such layered structures alongside centralized and decentralized ones as ways in which the agents of an LLM-based multi-agent system communicate. The hierarchy has a price: more model calls per task, hand-offs that can lose information, and the need to decide in advance how the task is split. For household scheduling the split follows the structure of the problem: which appliances can move, when each is cheapest, and how they fit under the peak limit together. The verification step between the specialists is where the independent check recommended by Jonah et al. (2026) fits.

2.5 Retrieval and solver tools for LLM agents

Two techniques extend what a language model agent can do without retraining it. Retrieval-augmented generation (RAG) gives the model relevant documents from an external store at the time of the request, so that its answer draws on knowledge it was not trained on and the store can be updated independently of the model (Lewis et al., 2020). For a planning task, the retrieved material can be solved examples of similar problems, which show the model what a good solution looks like. Whether such examples improve the search of a multi-step plan, rather than only its form, is an empirical question.

The second technique keeps the search out of the language model altogether. In OptiGuide, a language model answers questions about a supply chain by writing and running code against an optimization solver and explaining the result; the authors stress that the aim is to use solvers in tandem with language models rather than to replace them (Li et al., 2023). A household scheduling agent can be built the same way: the agents collect the household's constraints, call a solver, check its answer and explain it, while the solver guarantees the best schedule. Neither technique has been evaluated for household appliance scheduling. This thesis studies both as extensions of the pipeline (Section 4.4).

2.6 Agentic AI trends in 2025–2026

Three recent developments in agentic AI bear on this thesis. The first is the move from single agents to systems in which several LLM-based agents with distinct roles cooperate, a research area that has grown quickly since 2023 (Guo et al., 2024; Wang et al., 2024). Two surveys describe how the profiles, memory, planning and communication of such agents are organized, and they identify coordination between agents and the evaluation of multi-agent systems as open challenges. The idea is that a complex task can be divided among specialists, each with a smaller context and fewer tools than a single agent that handles the whole task.

The second is the Model Context Protocol (MCP), introduced by Anthropic in November 2024 and donated to the Linux Foundation's Agentic AI Foundation in December 2025, which has become a common standard for connecting agents to tools (Anthropic, 2024). MCP gives agents one model-independent interface to external tools and data sources, so each tool no longer needs its own integration code. The LangChain MCP adapters library makes MCP tools usable by LangGraph agents, so the price API and data reader of this thesis could be offered as reusable MCP servers.

The third is a growing set of design patterns for such agents. Liu et al. (2025) catalogue eighteen architectural patterns for agents built on foundation models, three of which match designs in this thesis: role-based cooperation, in which agents with assigned roles divide the work, describes the multi-agent pipeline; one-shot model querying, in which the model produces its answer in a single step, describes the LLM-without-tools baseline; and human reflection, in which humans give feedback on the agent's plan, describes a human-in-the-loop (HITL) checkpoint. Guidelines for human-AI interaction

likewise recommend that a system make clear why it did what it did and support efficient correction by the user (Amershi et al., 2019). The artifact therefore has an optional HITL checkpoint, a LangGraph interrupt node at which the Orchestrator shows the proposed schedule to the user for approval before it is finalized. The checkpoint was not enabled in the reported evaluations, which run without human intervention.

2.7 Finnish energy market context

Finland belongs to the Nordic spot electricity market (Nord Pool) and has priced electricity in 15-minute intervals since October 2025, which makes its residential market one of the most finely priced in Europe. At the end of 2024, about 33 % of all Finnish electricity contracts were dynamic-price contracts, up from 9 % in 2021; these pass hourly and quarter-hourly price changes straight to the consumer (Energy Authority, 2025).

Prices within a day vary a great deal. In the Nord Pool FI day-ahead prices from 2023 to 2026, the difference between the cheapest and the most expensive hour of a day had a median of 5.7 c/kWh excluding VAT and exceeded 10 c/kWh on 28 % of days. On days without near-zero prices the most expensive hour cost a median of 3.1 times the cheapest, and on 19 % of days at least one hour had a negative price (computed from Pakastin Oy, 2026). Extremes are far larger: the highest hourly price in the period, 189.6 c/kWh on 5 January 2024 at 19:00, was about one hundred times the median night-time price of 1.8 c/kWh. On 24 November 2023 the price fell to -500 EUR/MWh for half a day, and households with spot-price contracts noticeably increased their consumption (Haakana et al., 2025). At the same time, an estimated 7 to 15 % of Finnish households are energy poor (Aalto University, 2026). Shifting household demand in Finland therefore matters for the power system as well as for household bills.

2.8 Literature synthesis and research gap

Three findings come out of the reviewed literature. First, HEMS scheduling is a multi-step, multi-constraint optimization problem. Rules and optimizers solve it for the cases they were written or formulated for but do not explain their decisions, and language model agents have been proposed because they can take constraints stated in words and explain what they did (Section 2.1). Second, the first LLM-based HEMS implementations, both hierarchical and single-agent, have documented limitations in multi-appliance coordination, in scheduling on price alone, in the safety of committed schedules, and in evaluation scope (El Makroum et al., 2025; Jonah et al., 2026). Third, the existing home energy management tools that households can install range from cheapest-hours rules that plan each device alone to optimizers such as EMHASS, and retrieval of solved examples and solver tools are established ways to strengthen language model agents (Lewis et al., 2020; Li et al., 2023). No published HEMS study measures what an agent adds over these existing tools across several price patterns, or whether retrieval or a solver tool lets an agent reach the saving of an optimizer. Table 2.1 links

each limitation of the prior work to the design or evaluation choice of this thesis that responds to it, and to the research question that tests it.

Table 2.1 Limitations of prior work and how this thesis responds to them.

Limitation in prior work	Response in this thesis	Tested in
Multi-appliance coordination fails for some models (El Makroum et al., 2025; Jonah et al., 2026)	A Conflict Resolver, retrieval of solved days, an optimiser tool, and a household peak limit that forces appliances to be staggered on heavy-load days	SQ1, SQ2
Committed schedules can violate constraints the model reasoned about correctly; an independent check is recommended (Jonah et al., 2026)	A deterministic verification step checks every schedule for conflicts, missing appliances and window violations, and returns it for revision	SQ1 (validity)
Evaluation on a single day or a few days, one household, one price series (El Makroum et al., 2025; Jonah et al., 2026)	A factorial benchmark of four price patterns, two household loads and two peak limits, built from 18 REFIT households, and a held-out benchmark with pre-registered hypotheses	SQ2
Scheduling on the price signal alone (Jonah et al., 2026)	Partly addressed: every schedule must also keep base load plus appliances under a peak limit, with a strict and a realistic limit compared; the objective remains the electricity cost, and other signals such as carbon intensity are not added	SQ2
No comparison with existing home energy management tools	Comparison with cheapest-hours automations for Home Assistant and Shelly relays and with the optimizer EMHASS	SQ2
Rules and optimizers do not explain their decisions (Gokhale et al., 2024; Zharova et al., 2024)	The Orchestrator explains every recommendation and states the result of the final peak-load check	design

This thesis addresses that gap with three contributions. The first is the Multi-Agent Pipeline, a hierarchical ReAct multi-agent system in LangGraph that gives households daily appliance recommendations, using a library of solved days for retrieval and an optimiser tool; the design iterations that led to it are documented. The second is a measure of what its recommendations save, how often they are valid and what they cost to produce compared with a language model answering without tools (SQ1). The third is a map of the conditions, four price patterns, two household loads and two peak limits, under which the recommendations save more than, or as much as, existing home energy management tools, cheapest-hours automations and EMHASS (SQ2). The hypotheses were fixed in advance and tested on a held-out benchmark.

3. Research Methodology

This chapter describes the research methodology and explains how Design Science Research is applied in this thesis. It presents the rationale for the approach and the mapping of its phases to the thesis, the theories on which the design of the artifact draws, and the evaluation strategy: the systems compared, how the data were collected, and how they were analysed.

3.1 Design Science Research methodology

This thesis follows the Design Science Research (DSR) methodology as defined by Peffers et al. (2007). DSR designs and evaluates artifacts that solve practical problems. As Hevner et al. (2004) argue, design science creates and evaluates IT artifacts for identified organizational problems and produces knowledge by building and testing them. The main activity of this thesis is building and evaluating the multi-agent system, so DSR fits it.

Artifacts in DSR are classified into four types: constructs, models, methods, and instantiations (March & Smith, 1995). In this thesis, the artifact is an instantiation, that is, a working software implementation of a hierarchical multi-agent scheduling pipeline. It is supported by architectural models (the LangGraph state graph design) and constructs (the agent role taxonomy: Profiler, Scheduler, Resolver, Orchestrator). Table 3.1 maps the six DSR phases defined by Peffers et al. (2007) to the activities and outputs of this thesis.

Table 3.1 DSR phase mapping.

Phase	DSR step	Activity in this thesis	Output
1	Problem identification	SLR confirming no multi-agent ReAct HEMS addresses multi-appliance coordination across multiple price patterns	Gap statement; confirmed RQs
2	Define objectives	Specify measurable success criteria: saving against no scheduling, validity, conflict resolution rate, latency, token use	Objective specification
3	Design and development	Implement Appliance Profiler, Price-Aware Scheduler, Conflict Resolver and Orchestrator in LangGraph; add a case library for retrieval and an optimiser tool	Working multi-agent artifact
4	Demonstration	Run pipeline on five representative scenarios; confirm valid cost-reducing schedules	Demonstration results
5	Evaluation	Exploratory run on 48 hourly-price scenarios; two pre-registered confirmatory runs on held-out fifteen-minute scenarios (Q, Q2); comparison with existing home energy management tools; Wilcoxon and TOST tests	Quantitative evaluation results
6	Communication	Write findings, design guidelines, limitations, and future work	Complete thesis

3.2 Theoretical foundations

Design science research grounds the design of an artifact in kernel theories, the existing theories from which its design principles are derived, and positions its contribution against them (Gregor & Hevner, 2013). Four bodies of work serve this role here. The first is demand flexibility under time-varying prices: a household lowers its cost by moving flexible loads to low-price periods, within the time windows its members accept and the power limits of its connection. This is the problem that rule-based and optimization-based HEMS address (Section 2.1), and the MILP formulation of Sou et al. (2011) states it precisely. It defines what a good recommendation is: the cheapest schedule that respects every window and the peak limit.

The second is reasoning grounded in tool observations. ReAct (Yao et al., 2023) holds that a language model solves multi-step tasks more reliably when it alternates between reasoning and actions whose results it observes, instead of producing an answer from its parameters alone. It yields the design principle that every number an agent uses comes from a deterministic tool, and the LLM without tools is the baseline that tests it (H1a). The third is task decomposition with role specialization in multi-agent systems (Hong et al., 2024; Guo et al., 2024): a task split into specialist roles with structured hand-offs keeps each agent's context and tool set small and makes intermediate results checkable (Section 2.4). It yields the four roles of the pipeline and the verification step between

them.

The fourth is the reuse of solved problems. Case-based reasoning solves a new problem by retrieving similar past cases and adapting their solutions (Aamodt & Plaza, 1994), and retrieval-augmented generation supplies a language model with such material at the time of the request (Lewis et al., 2020). Together with the use of a solver as a tool (Li et al., 2023), it yields the case library and the optimiser tool of the final design. Guidelines for human-AI interaction (Amershi et al., 2019) add the requirement that the system explain what it did and state when a constraint is not met, which the Orchestrator's summary implements.

The evaluation follows the Framework for Evaluation in Design Science Research (FEDS) of Venable et al. (2016). All evaluations are artificial, on simulated household days, and they move from formative to summative. The design iterations were formative evaluations on an exploratory benchmark, and the pre-registered run on a held-out benchmark is the summative evaluation. Of the strategies FEDS describes, this corresponds to Technical Risk and Efficacy, which is suited to an artifact whose main risk is whether it works technically before it is evaluated with users. A naturalistic evaluation with real households is left to future work (Section 6.3).

3.3 Evaluation strategy

The evaluation is a controlled simulation that needs no sensors or hardware. Its inputs come from public datasets and APIs, and every scenario is built from measured appliance data and historical prices. The comparison is paired: every system receives the same scenarios and is scored by the same function.

Systems compared. The artifact is the *Multi-Agent Pipeline* with GPT-4o-mini (Chapter 4). Its Scheduler and Resolver receive the two most similar solved days from a case library as worked examples, and the Scheduler has a tool that solves the whole day exactly; the agents decide when to call it, check its answer and explain the result (Section 4.4). Its earlier versions, without the optimiser tool and without retrieval, are design iterations: they were run on the exploratory benchmark and are reported in Section 5.3.1, but they are not compared systems. The pipeline is compared with the following alternatives:

- *No scheduling*: every appliance starts as soon as the user allows. It defines the saving of every other system.
- *Cheapest-hours automation*: the selection rule of the Home Assistant blueprint by Kert (n.d.). Within the appliance's window, the prices are sorted in ascending order and the threshold is the price at index $4h$ (counting from zero), where h is the cycle length rounded up to whole hours. The blueprint switches a device on whenever the price is at or below the threshold; for a cycle appliance, it is assumed that this trigger

starts the cycle, which then runs to completion. With hourly prices, the same rule describes the scattered cheapest-hours mode of shelly-porssisahko (jisotalo, n.d.).

- *Cheapest window*: the block of whole hours, as long as the cycle rounded up, with the lowest average price between the ready time and the deadline. This is how the Nordpool Planner (dala318, n.d.) and the sequential mode of shelly-porssisahko choose when to switch a device on.
- *Smart timer (own heuristic)*: a peak-aware rule written for this study as a reference point, not a deployed product. It shows how far a simple rule gets without a language model or optimisation, and a household energy application could implement it. It places the appliances one at a time, the most energy-intensive first, each at its cheapest start at which the household load stays within the peak limit. The rule was fixed before it was run on the benchmark and was not tuned afterwards.
- *EMHASS* 0.18.4, run as households run it: as a Docker service with its default settings, called through its REST interface. Each scenario is sent as runtime parameters of a day-ahead optimisation: the prices, the base load as the load forecast, no photovoltaic production, the appliances as deferrable loads with their time windows and measured power profiles, and the peak limit as the maximum power drawn from the grid. Home Assistant itself is not involved, since all data is passed with the request. A configuration with a constant nominal power per appliance, set to the cycle's peak, is reported as a secondary result. As a cross-check, the same scenarios were also solved by calling EMHASS's optimisation module directly with an exact solver tolerance; the two routes give the same schedule in most scenarios, and the remaining differences stem from the default tolerance of the service.
- *LLM without tools*: the same model as the agents, given the day's appliance plan together with all household data and the price and measured base load of every slot in a single prompt. Its schedule is taken from one answer, without tools or a reasoning loop. This corresponds to the standard prompting baseline of Yao et al. (2023) and isolates the contribution of tool use.

The cheapest-hours rules plan each appliance on its own and ignore the household's total load, exactly as the real tools do. Because language model output varies between runs, each system that uses a language model is run three times per scenario, while the deterministic systems are run once.

The MILP solver of Section 4.5 is not a compared system. It confirms during benchmark construction that every scenario has a feasible schedule, it solves the days of the case library, and it is the solver behind the optimiser tool. Since a schedule cannot save more than the MILP schedule, its saving is reported as the upper bound in the result tables.

Stages. The system was developed on a first benchmark of 48 scenarios with hourly prices in six scenario types, over the design iterations of Section 5.3.1. A first confirmatory

run, on 48 held-out scenarios with fifteen-minute prices (the Q set, Section 5.2), tested hypotheses registered in advance about the pipeline, the LLM without tools and an earlier single-agent design. Its results are reported in Section 5.7 and remain valid, but the thesis no longer builds its questions on the single-agent comparison.

The research design was then revised. The single agent was dropped as a compared system, the saving against no scheduling replaced the gap to the optimum as the primary outcome, retrieval and the optimiser tool were added, and the cheapest-window rule joined the existing tools. The six scenario types were replaced by a factorial design in which the price pattern, the household load and the peak limit vary separately (Section 5.1), because each of the old types changed several of these at once. The pre-registration records these changes as a dated amendment. Every system was then run on a new exploratory benchmark of 96 scenarios with this design. The hypotheses of Table 3.2 were fixed afterwards in a second pre-registration, committed to the project repository before any run on a second held-out benchmark of 96 scenarios (the Q2 set), whose days appear in no other benchmark and not in the case library. Only the run on Q2 is used to accept or reject them.

Table 3.2 Hypotheses and statistical tests (second pre-registration, Q2 set).

	Hypothesis	Test
SQ1, value and cost		
H1a	The Multi-Agent Pipeline saves more than the LLM without tools.	Wilcoxon signed-rank
H1b	Its recommendations are valid more often than those of the LLM without tools.	Wilcoxon signed-rank
H1c	Its mean response time is below 60 s and its mean API cost below \$0.10 per recommendation (two tests).	One-sided t test against the limit
SQ2, conditions (over all price patterns, loads and limits)		
H2a	The Multi-Agent Pipeline saves more than the cheapest window.	Wilcoxon signed-rank
H2b	It saves more than the cheapest-hours automation.	Wilcoxon signed-rank
H2c	It saves the same as EMHASS within ± 1 percentage point.	Paired t TOST

3.3.1 Data collection

The study uses secondary data only; no new measurements were taken and no human participants were involved. Two kinds of input data are used. Appliance cycles, base loads and fixed appliances come from the REFIT Electrical Load Measurements dataset, which records appliance-level power in 20 UK households at eight-second resolution between 2013 and 2015 and is published under an open licence with the households anonymised (Murray et al., 2017). Day-ahead electricity prices for the Nord Pool FI bidding zone come from two public services that republish Nord Pool data, Sähkötin and Elering, and were checked against the official prices of the ENTSO-E Transparency Platform (Section 5.1). Both are combined into benchmark scenarios by a fixed, seeded procedure, described in

Section 5.1, so that every benchmark can be rebuilt exactly.

The evaluation data are generated by running the systems on these scenarios. Every run is stored as one record in a results file. The record contains the schedule, its cost, whether it is valid and which constraint it violates if not, the number of input and output tokens, the API cost, the response time, the number of tool calls and feedback rounds, and any error the agents reported. The reasoning trace and tool calls of every language model run are also recorded with LangSmith. EMHASS is called through its REST interface, and its schedules are scored by the same function as all other systems. Runs are resumable, so an interruption repeats only the run in progress, and a run that failed because of a provider or network error is retried and not recorded. Every completed run is kept, including invalid ones.

3.3.2 Data analysis

Scoring

All schedules are scored by the same function. The cost of a schedule is the consumer price of electricity multiplied by the energy of the flexible appliance cycles, summed over the day; the consumer price adds the retail margin, distribution fee, electricity tax and VAT to the spot price (Section 5.1). The base load costs the same under every schedule and is therefore excluded from the cost, but it counts towards the peak load limit. A schedule is valid when every appliance has a start time, every cycle runs within its user window, and the total household load never exceeds the peak limit. An invalid schedule, or a run that returns no schedule, is scored at the cost of no scheduling, on the reasoning that the household would then fall back to starting its appliances when they are ready. The validity rate is reported separately, so that this convention does not hide failures.

The primary outcome is the saving against no scheduling:

$$\text{saving}(\%) = \frac{\text{cost without scheduling} - \text{system cost}}{\text{cost without scheduling}} \times 100.$$

It is what a household gains by following the recommendation, and it needs no optimum as a reference. Because the consumer price stays positive even when the spot price is negative, the cost without scheduling is always positive. Over a set of runs, the pooled saving divides the summed savings by the summed costs without scheduling, so that expensive days weigh more than cheap ones, as on a bill; the paired tests use the per-scenario saving. An invalid run saves nothing. The gap to the MILP optimum, $(\text{system cost} - \text{MILP cost})/\text{MILP cost} \times 100$, is reported as a secondary measure, because it shows how much saving a system leaves on the table.

The choice of tariff does not change the ranking of the systems. The margin, distribution fee and electricity tax are constant per kilowatt-hour, and every cycle uses the same energy wherever it is placed, so the consumer price changes the cost of every schedule by the same amount and scales all cost differences by the common factor $1 + v$. The ranking

of the systems and the euro differences between them are therefore the same as under the spot price, apart from that factor, which was also verified on the recorded schedules. The saving in percent does depend on the tariff, through its denominator: under the spot price alone, the same euro saving would be a larger share. The consumer price is used because it is what the household pays.

The sub-questions use these measures as follows. SQ1 compares the saving and validity of the pipeline with those of the LLM without tools, and checks its response time and API cost per recommendation against the design objectives of Section 4.1 (60 s and $0.10); tokens (input plus output) are reported alongside. SQ2 compares the saving and validity of the pipeline with those of the existing home energy management tools, the cheapest-hours rules and EMHASS, first over all scenarios and then separately for each price pattern, household load and peak limit. The breakdown by condition is descriptive; the registered tests concern all scenarios together. Also reported without hypotheses are the total euro saving, the smart timer (own heuristic), the gap to the optimum, and the conflict resolution rate, which is the share of valid runs among the scenarios that require coordination (those in which starting every appliance at its individually cheapest time would exceed the peak limit).

Statistical analysis

Two systems are compared per scenario, using the mean saving over the runs as the paired observation. The Wilcoxon signed-rank test assesses whether the difference across the 96 paired scenarios is statistically significant. It is designed for paired comparisons and does not assume that the differences are normally distributed. Pratt's treatment of zero differences is used, so that scenarios in which both systems reach the same cost remain in the ranking instead of being discarded. Practical significance is reported with the effect size $r = |Z|/\sqrt{N}$. All tests are two-sided at $\alpha = 0.05$. A directional hypothesis is supported when its test is significant and the effect has the predicted direction. Two-sided tests are the conservative choice for directional hypotheses, since a one-sided test would halve each p-value. The p-values of the registered hypotheses are adjusted for multiple testing with the Holm procedure within each sub-question: four tests for SQ1 and three for SQ2. The two tests of H1c compare a mean with a fixed limit, which is a one-sided claim by nature, so they are one-sided: the upper end of the one-sided 95 % confidence interval must lie below the limit.

A non-significant difference does not by itself show that two systems perform alike, since it can also result from too little data. Equivalence with EMHASS (H2c) is therefore tested with the two one-sided tests (TOST) procedure (Schuirmann, 1987; Lakens, 2017). The systems are considered equivalent when both one-sided tests reject, at the 5 % level, a mean difference in saving of at least one margin in either direction. This is the same as requiring the 90 % confidence interval of the mean paired difference to lie inside the margin. The margin is ± 1 percentage point of the cost without scheduling. On the Q2 set that cost has a median of € 0.61 per day (interquartile range € 0.28 to € 1.06, maximum € 2.65), so one percentage point is about 0.6 cents on the median day and under three cents

on the most expensive one, which a household would not notice on its bill. The paired t version of the test concerns the mean difference and is the primary one; a Wilcoxon version, which concerns the typical scenario, and margins of 0.5 and 2 percentage points are reported alongside.

4. System Design and Implementation

This chapter describes the design and implementation of the hierarchical ReAct multi-agent system, the Multi-Agent Pipeline. It presents the problem definition and design objectives, the architecture of the pipeline, the specifications of each agent, the retrieval of solved days and the optimiser tool, the MILP solver, and the technology stack used throughout.

4.1 Problem definition and objectives

Based on the literature synthesis in Section 2.8 and DSR Phase 2, the artifact must satisfy five design objectives. First, multi-appliance coordination: the pipeline must correctly schedule two or more flexible appliances simultaneously without producing overlapping high-load windows. Second, price-volatility robustness: the pipeline must produce cost-reducing schedules under every price pattern, household load and peak limit of the benchmark. Third, constraint compliance: every schedule must respect user-defined completion deadlines and household load limits. Fourth, interpretability: the full reasoning trace of every scheduling decision must be captured through LangSmith. Fifth, computational efficiency: the pipeline must complete scheduling within 60 seconds and under $0.10 API cost per scenario using GPT-4o-mini.

4.2 Architecture overview

The artifact is a LangGraph state graph in which each node is either a ReAct agent or a coordination step. All nodes read from and write to a shared state object that persists for the whole run (Table 4.1). The Orchestrator first turns the household's request into a task brief listing the appliances to schedule. The Appliance Profiler then classifies the appliances and records the power draw, cycle duration and time window of every flexible one. The Price-Aware Scheduler proposes a start time for each flexible appliance from the price curve, and the Conflict Resolver checks the proposal against the household peak load limit and staggers start times where appliances overlap.

After the Resolver, the Orchestrator's verification step checks the resolved schedule for three kinds of problems: remaining peak load conflicts, detected with the same conflict tool that the Resolver uses; requested appliances that have no start time; and start times outside the user's time window. If a problem remains, the step sends a description of it back to the Scheduler, which proposes a revised schedule for the Resolver to check again. The Scheduler and Resolver run at most twice, that is, with one feedback round. The schedule is then finalised, and the Orchestrator writes a short summary for the household that states the result of the final peak load check. The original request travels in the shared

state together with the Orchestrator's brief, so an appliance missing from the brief can still be profiled and scheduled. Figure 4.1 shows the architecture, and Table 4.1 lists the fields of the shared state.

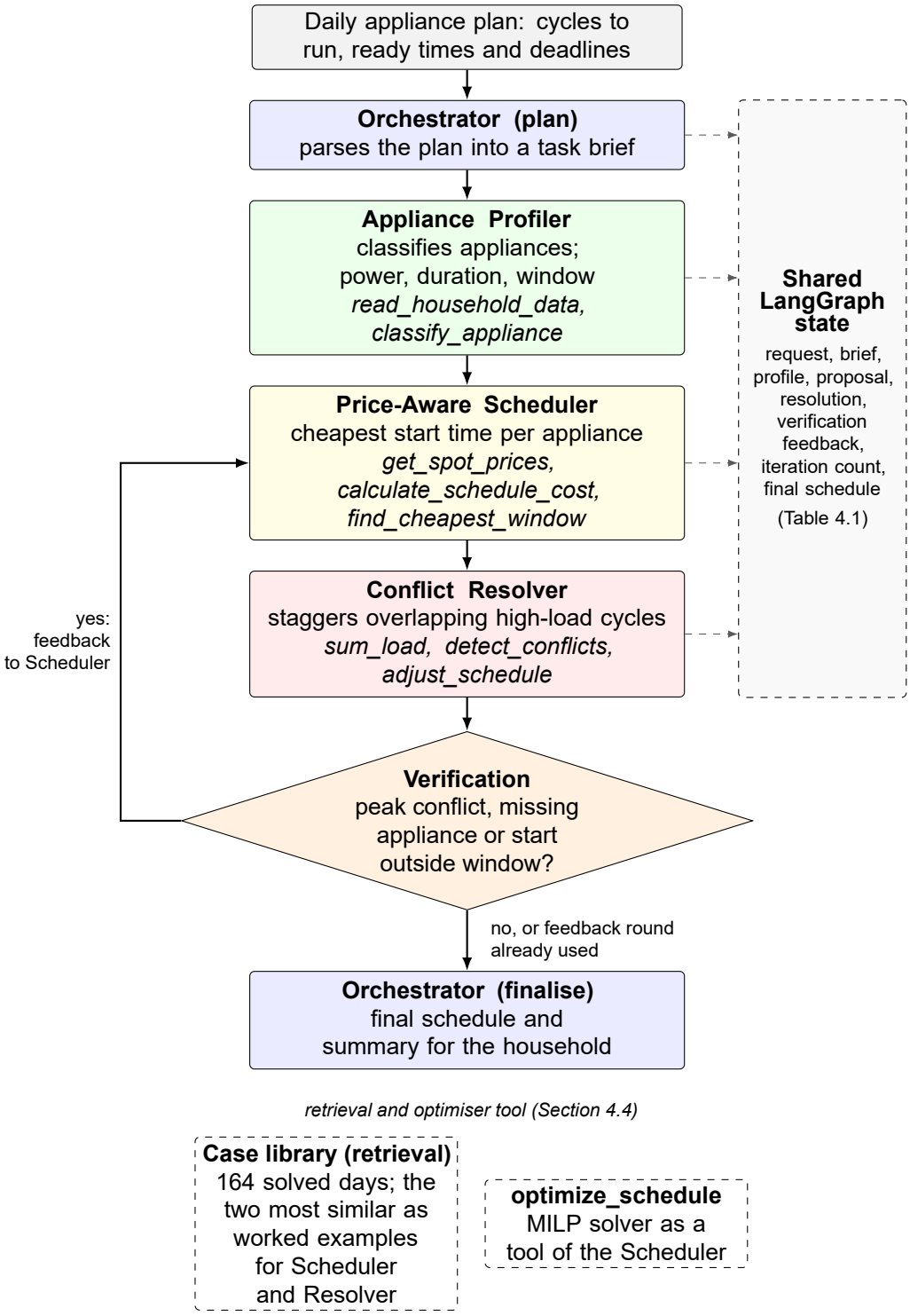


Figure 4.1. Architecture of the hierarchical ReAct multi-agent system.

Table 4.1 Fields of the shared LangGraph state.

Field	Written by	Content
request	caller	the household's scheduling request in natural language
brief	Orchestrator	appliance identifiers to schedule and stated constraints
profile	Profiler	per appliance: type, flexible or fixed, peak power, cycle duration, earliest start and deadline
proposal	Scheduler	start time per appliance, estimated cost and a short justification
resolution	Resolver	conflict-free start times, whether conflicts remain, and the changes made
verification feedback	verification step	remaining peak conflicts, appliances without a start time and start times outside the user window; empty when the schedule passes
iterations	verification step	number of Scheduler–Resolver passes completed (at most two)
final, summary	Orchestrator	final start times and the summary shown to the household
tool calls, recoveries, errors	all nodes	counters and error messages recorded for the evaluation

4.3 Agent specifications

The **Orchestrator** receives the household scheduling request, coordinates the three specialist agents in sequence, and writes the final output. It does no scheduling itself, so its context stays small.

The **Appliance Profiler** reads the household consumption data and classifies each appliance by shiftability. The classification distinguishes flexible loads, which have long cycle durations and user-defined completion windows (washing machines, dishwashers and tumble dryers), from fixed loads such as refrigerators and microwaves. The Profiler outputs a structured appliance profile containing power draw in Watts, typical cycle duration in minutes, and the constraint window for each flexible load.

The **Price-Aware Scheduler** receives the appliance profile and the day-ahead prices. It finds the cheapest window for each flexible appliance within its user window and proposes a daily schedule with a start time per appliance and an estimate of the total cost.

The **Conflict Resolver** receives the proposed schedule and looks for times when two or more appliances with high power draw would run together and exceed the household's peak load limit. If it finds such a conflict, it staggers start times to remove it, choosing the moves that add the least to the total cost.

Each specialist is a ReAct agent that may only call its own tools and must end with a

structured output (the appliance profile, the proposed schedule or the resolved schedule), which is validated against a schema before it is written to the shared state. The agents share a budget of 75 graph steps, divided by workload: 15 for the Profiler, 35 for the Scheduler and 25 for the Resolver. The Scheduler received the largest share after the pilot runs showed that it performs most of the tool calls. When a model replies in prose instead of producing the required structured output, the agent is resumed from its last state and asked once more to return the output, at most twice. This rule applies identically to every agent and model, and every such recovery is recorded.

Table 4.2 lists the tools. They are deterministic functions over the scenario data, so every number an agent reasons with comes from a calculation rather than from the language model. The first seven tools each handle part of the problem: `find_cheapest_window`, for example, considers one appliance at a time and ignores both the other appliances and the peak load limit. With these tools alone, coordinating the appliances is left to the agents. The eighth, `optimize_schedule`, solves the whole day and was added in the last design iteration (Section 4.4).

Before the optimiser tool was added, the automatic mode of `adjust_schedule` was the agents' only way to repair a conflict. It optimizes the start of one appliance while all others stay where they are and never re-plans several appliances together. Which appliances to move, in which order, and when to stop were decisions of the agent, and a poor sequence of single moves could settle on a schedule far from the optimum. In scenario D06 of the first benchmark, for example, the pipeline ended about 30 % above the optimal cost with a staggered schedule that no single further move could improve. This limitation led to the retrieval of solved days and the optimiser tool described next.

Table 4.2 Tools available to the agents.

Tool (inputs)	Agent	Output and logic
`read_household_data()`	Profiler	Peak load limit, measured base load range, and for each appliance its type, cycle duration, energy, peak power and user window (flexible) or rated power (fixed).
`classify_appliance(appliance_id)`	Profiler	Flexible or fixed, with a reason; washing machines, dishwashers and tumble dryers are shiftable cycle loads.
`get_spot_prices(from_time, to_time)`	Scheduler	Consumer price in c/kWh for every 15-minute slot, optionally for a time range.
`calculate_schedule_cost(appliance_id, start_time)`	Scheduler	Cost of one cycle started at the given time (power profile $\times$ price $\times$ 0.25 h, summed over the cycle), its end time, and whether it lies within the user window.
`find_cheapest_window(appliance_id, top_k)`	Scheduler	The k cheapest start times within the user window for one appliance, found by evaluating every valid start.
`sum_load(schedule)`	Resolver	Maximum total household load (base load plus scheduled cycles) and the five highest slots, compared with the limit.
`detect_conflicts(schedule)`	Resolver	Time ranges in which the total load exceeds the limit and the appliances running then, or confirmation that there is no conflict.
`adjust_schedule(schedule, appliance_id, new_start_time)`	Resolver	Moves one appliance and reports the cost change, the window check and any remaining overload. Without a start time, it moves the appliance to its cheapest start at which it causes no overload, keeping the others fixed, and lists alternatives.
`optimize_schedule(appliance_ids)`	Scheduler*	Solves the day for the given appliances with the MILP of Section 4.5 and returns the start times, the cost and the maximum load. *Added in the last design iteration.

4.4 Retrieval of solved days and the optimiser tool

Case library. Retrieval gives the agents examples of how similar days were solved best. The case library holds 320 cases: 20 price days for each combination of price pattern and household load, each under both peak limits. The days are drawn from the hourly prices up to 30 September 2025, excluding every day of every benchmark, so no evaluated day is in the library. Each case is composed exactly like a benchmark scenario (Section 5.1), with the same REFIT cycle pool, price patterns, loads and limits, and solved with the MILP of Section 4.5. A deployed system would build such a library from the household's own past days in the same way.

Retrieval. Each day is described by two vectors: the shape of its price curve (24 hourly consumer prices, standardised to mean zero and unit variance) and six structural values (the number of appliances, their combined peak power relative to the headroom under the limit, the mean window length, the share of appliance pairs whose windows overlap, whether the individually cheapest starts would exceed the limit, and the relative price spread). The similarity of two days is the mean of the cosine similarity of the price shapes and e^{-2d}, where d is the Euclidean distance between the structural values. The two most similar cases are retrieved. The description and the weights were set once, before any evaluation run, and were not tuned. The retrieval is deterministic and needs no embedding model.

Worked examples. Each retrieved case is turned into a short text in the agents' own vocabulary: the number of appliances, the peak limit and the price range; whether the individually cheapest starts would exceed the limit; the optimal start of every appliance with its own cheapest start and the extra cost of the difference; and the pattern of the staggering, for example that the tumble dryer starts as the washing machine ends. The text is appended to the messages of the Scheduler and the Resolver with an instruction to use the examples as patterns, not as answers, and to check the day's own prices and loads with the tools.

Optimiser tool. The Scheduler also has `optimize_schedule` (Table 4.2), and its prompt asks it to call the tool once with all flexible appliances, to check any unusual start time with `calculate_schedule_cost`, and to fall back to `find_cheapest_window` if the solver finds no schedule. The Resolver and the verification step are unchanged, so every schedule is still checked independently of the tool that produced it, as Jonah et al. (2026) recommend. This follows the division of work of Li et al. (2023): the solver searches, and the agents formulate the call, check the result and explain it to the household.

The pipeline reached this form over several design iterations (Section 5.3.1): the base pipeline without retrieval or the optimiser tool, then the pipeline with retrieval only. Both earlier versions were run on the exploratory benchmark, and their results show what each addition contributed. An earlier version of the study also compared the base pipeline with a single ReAct agent that had the first seven tools of Table 4.2 and the same step budget; that comparison is summarised in Section 5.7. Only the final Multi-Agent Pipeline is a compared system in the current evaluation.

4.5 MILP solver

The MILP solver computes the optimal schedule of a day. It is used in three roles: it confirms during benchmark construction that every scenario has a feasible schedule, it solves the days of the case library, and it is the solver behind the optimiser tool. Its schedules also give the upper bound on the saving in the result tables. The model is

the standard time-indexed formulation of appliance scheduling with binary start variables (Sou et al., 2011). The day is divided into slots $t \in \mathcal{T} = \{0, \dots, 95\}$ of $\Delta = 0.25\,\text{h}$. Each flexible appliance $a \in \mathcal{A}$ has a power profile $p_{a,0}, \dots, p_{a,d_a-1}$ in kW over its cycle of d_a slots, an earliest start slot e_a and a deadline slot ℓ_a by which the cycle must end. Its admissible start slots are

$$\mathcal{S}_a = \{\, s \in \mathbb{Z} : e_a \le s \le \ell_a - d_a \,\}.$$

The model uses binary start variables $x_{a,s} \in \{0, 1\}$, equal to one if appliance a starts in slot $s \in \mathcal{S}_a$. With the consumer price π_t in €/kWh, the base load b_t in kW and the household peak limit $\bar{P}$ in kW, the model is

$$\min_{x} \quad \sum_{a \in \mathcal{A}} \sum_{s \in \mathcal{S}_a} c_{a,s}\, x_{a,s}, \qquad c_{a,s} = \Delta \sum_{k=0}^{d_a-1} p_{a,k}\, \pi_{s+k}, \tag{4.1}$$

$$\text{s.t.} \quad \sum_{s \in \mathcal{S}_a} x_{a,s} = 1 \qquad \forall a \in \mathcal{A}, \tag{4.2}$$

$$b_t + \sum_{a \in \mathcal{A}} \sum_{\substack{s \in \mathcal{S}_a \\ 0 \le t-s < d_a}} p_{a,t-s}\, x_{a,s} \le \bar{P} \qquad \forall t \in \mathcal{T}, \tag{4.3}$$

$$x_{a,s} \in \{0, 1\} \qquad \forall a \in \mathcal{A},\ s \in \mathcal{S}_a.$$

The objective (4.1) is the consumer cost of the flexible cycles, with $c_{a,s}$ the precomputed cost of starting appliance a in slot s. Constraint (4.2) requires exactly one cycle per appliance, and constraint (4.3) keeps the total household load within the peak limit in every slot. Because a decision fixes the start of a whole cycle, each cycle runs without interruption once started, and restricting the variables to $\mathcal{S}_a$ keeps every cycle within its user window, so neither property needs a separate constraint. The model is solved with PuLP and the CBC solver in well under a second per scenario. Its correctness was verified against exhaustive enumeration of all start combinations on small scenarios.

4.6 Technology stack

Table 4.3 lists the software used to build and evaluate the artifact, with the versions used for the reported results. All components except the commercial language model are open source. The API cost of the commercial model is an outcome of the evaluation and is reported with the results in Chapter 5.

Table 4.3 Technology stack.

Component	Tool and version	Purpose
Programming language	Python 3.12	implementation of the artifact, baselines and analysis
Agent orchestration	LangGraph 1.2, LangChain 1.4	state graph of the pipeline, ReAct agents, tool calling and schema-validated outputs
Commercial LLM	GPT-4o-mini (snapshot 2024-07-18), OpenAI API	model of the Multi-Agent Pipeline, its design iterations and the LLM without tools
Open-weight LLM	gpt-oss-20b, Ollama 0.34 and 0.35	open-weight model hosted on local hardware, used in an exploratory run of the earlier design only; Ollama updated itself from 0.34.4 to 0.35.0 during that run
MILP solver	PuLP 3.3 with CBC	benchmark feasibility check, case library and the optimiser tool
HEMS optimizer	EMHASS 0.18.4, Docker	open-source home energy management optimizer compared in SQ2, called through its REST interface
Data processing	pandas 3.0, NumPy 2.5	REFIT cycle extraction, price series and scenario construction
Tracing	LangSmith	recording the reasoning trace and tool calls of every run
Statistics	SciPy 1.18	Wilcoxon signed-rank test, TOST equivalence test, effect size and Spearman correlation

5. Evaluation and Results

Draft note: the results sections of this chapter still report the earlier design comparison with the single agent and the optimality gap. They will be revised with the main-benchmark results of retrieval and the optimiser tool and with the second confirmatory run on Q2 (Section 3.3).

This chapter presents the evaluation of the artifact, corresponding to DSR Phases 4 and 5. Section 5.1 describes the benchmark: the appliance and price data and the construction of the 48 evaluation scenarios across six price-volatility types. Section 5.2 uses the MILP solver alone to examine Finland's move to fifteen-minute electricity prices, quantifying how much of the achievable scheduling saving depends on fifteen-minute rather than hourly price information; this establishes where differences between schedulers can be expected before any agent is evaluated. Section 5.3 reports the demonstration of the pipeline on representative scenarios (DSR Phase 4). Sections 5.4 and 5.5 compare the optimality gaps of the multi-agent pipeline and the single-agent baseline across all scenarios and test the difference with the Wilcoxon signed-rank test. Section 5.6 compares both designs with two simpler baselines, a rule-based smart timer (own heuristic) and a language model without tools, and with two real home energy management tools, the EMHASS optimizer and a Home Assistant cheapest-hours automation. Section 5.7 repeats the tests of H1 and H2 on the held-out fifteen-minute benchmark, as fixed in the pre-registration. Section 5.8 breaks the cost savings down by appliance type. The agent results in Sections 5.3–5.8 are pending the full evaluation runs.

5.1 Dataset and scenario design

The evaluation benchmark combines real appliance consumption profiles from UK households with real Finnish day-ahead electricity prices. Each of the 48 scenarios describes one day in 96 fifteen-minute slots (00:00–24:00 local time) and consists of a price curve, a household base load, a set of flexible appliance cycles with user time windows, and a household peak load limit. The benchmark is generated by a deterministic, seeded build procedure, so it can be reproduced exactly from the source data.

5.1.1 Appliance data

Appliance profiles are drawn from the cleaned REFIT Electrical Load Measurements dataset (Murray et al., 2017; Murray & Stankovic, 2016), which records the whole-house aggregate and nine individually monitored appliances at 8-second intervals in 20 UK households between 2013 and 2015. The cleaned release corrects timestamps for daylight saving time and re-aligns appliance monitor columns where a monitor was moved between appliances during the study. The appliance-to-column mapping follows the

dataset documentation.

Eighteen households are used. House 12 is excluded because it has no monitored washing machine, dishwasher or tumble dryer. House 19 is excluded after inspection: its washing-machine monitor recorded no heating phase in any of its 246 detected cycles (no fifteen-minute mean above 1.2 kW, against 22–70 % of cycles in the other households), so its profiles do not represent the water-heating load that dominates the cost and peak demand of a wash. Washer-dryers (houses 1, 8, 9 and 18) are not used as profiles, because they run washing and drying as a single cycle; their consumption is nevertheless removed from the base load.

Individual appliance cycles are extracted from each monitor by thresholding the one-minute mean power at 20 W and merging pauses shorter than 30 minutes for dishwashers (whose programmes contain a passive drying pause) and 15 minutes for washing machines and tumble dryers (which are often reloaded within minutes). Cycles shorter than 20 minutes, longer than 300 minutes or below 0.1 kWh are discarded, and each remaining cycle is converted into a fifteen-minute mean power profile. This yields 6,172 washing-machine cycles from 19 monitored machines, 4,702 dishwasher cycles from 15 machines and 1,940 tumble-dryer cycles from 7 machines. To prevent heavily used appliances from dominating the benchmark, a scenario first draws a monitored appliance of the required type and then one of its cycles. The household base load is the median, per fifteen-minute slot of the day, of the aggregate minus all shiftable appliance monitors, computed after discarding readings that the dataset flags as inconsistent (sub-metered sum exceeding the aggregate).

5.1.2 Electricity price data

Day-ahead spot prices for the Nord Pool FI bidding zone come from two public services that republish Nord Pool data. The hourly prices of the exploratory benchmark and the case library, 1 January 2022 to 30 September 2025, come from Sähkötin (Pakastin Oy, 2026); each hourly value is repeated across the four fifteen-minute slots of its hour. The fifteen-minute prices of the held-out benchmarks, from 1 October 2025, come from Elering (Elering AS, 2026). All values are in EUR/MWh excluding VAT, and days with a daylight-saving transition are dropped. Both sources were checked against the official day-ahead prices of the ENTSO-E Transparency Platform for the whole period (ENTSO-E, 2026). The Elering prices are identical to the official ones at all 67,411 timestamps. The Sähkötin prices are identical for every hour before October 2025 except one hour on a daylight-saving transition day, which no benchmark uses. Every price in the scenarios of the exploratory and the held-out benchmarks therefore matches the official data.

Scheduling cost is computed with the consumer price rather than the raw spot price. For spot price s, the consumer price per kWh is

$$p = (s + m + d + \tau)(1 + v),$$

with a retail margin $m = 0.5$ c/kWh, a distribution fee $d = 5.0$ c/kWh, the electricity tax $\tau = 2.253$ c/kWh and VAT $v = 25.5\,\%$. These are representative values for a Finnish household on a spot-price contract. The tax is that of tax class I including the security-of-supply fee, valid until 31 March 2026; on 1 April 2026 the fee rose from 0.013 to 0.085 c/kWh, raising the tax to 2.325 c/kWh (Caruna, 2026). The benchmark keeps one tariff for all days, including a single VAT rate, so that scenarios from different periods remain comparable. Because every component is constant per kilowatt-hour, the choice of values does not change the optimal schedule, the ranking of the systems or the savings captured, only the size of the optimality gap (Section 3.3). Using the consumer price keeps costs positive on days with negative spot prices, which keeps the optimality gap defined in Section 3.3 meaningful. Only the energy of the flexible appliances is costed, since the base load costs the same under every schedule.

5.1.3 Scenario construction

Table 5.1 summarises the six scenario types. Days are assigned to types by the within-day standard deviation of the spot price. Types A and B use disjoint days from the lowest quartile among days without negative prices, type C uses days between the 40th and 60th percentiles, and type D uses days in the top 15 % that fall in December–February. Type E uses days from December 2022, sampled from the 16 days with the highest maximum price. Type F uses days with at least two hours of negative prices. Eight days are sampled per type.

Types C–F need up to five loads, while REFIT provides three flexible appliance types; a scenario may therefore contain two cycles of the same type, such as two washing loads. Each appliance has a user window (earliest start and deadline), and its naive start time is the earliest start. Types A–C use windows opening between 06:00 and 12:00 and lasting 8–14 hours. In type D all five appliances share one window from 08:00–10:00 until 22:00–24:00, so their cheapest start times coincide. In type E each appliance becomes ready 0–2 hours before the day's highest daytime price and has 4–8 hours of slack beyond its cycle, so the naive schedule runs into the price spike while avoiding it remains possible. This reflects that price spikes coincide with evening demand peaks, when households tend to start appliances. Type F windows open between 00:00 and 04:00 and last until midnight.

The household peak load limit, including the base load, is 3.5 kW in every fifteen-minute slot. It is a deliberately strict, stylized cap that makes the appliances compete for capacity, rather than a limit that Finnish households face today. The main fuse (typically 3×25 A, about 17 kW) would never bind for these appliances. Power-based distribution tariffs exist, but they price the peak rather than forbid it. Helen Sähköverkko's time-based distribution product, for example, charges €1.57 per kW and month, excluding VAT, for the third-highest hourly average power of the month, with 80 % of night-time power counted (Helen Sähköverkko, 2026a). Under the Energy Authority's guidance, which network companies must follow by 1 January 2029 at the latest, a household power charge is based on the highest 60-minute average power of the month, starts only above 8 kW, and remains optional, since a product without a power charge must still be offered (Helen

Sähköverkko, 2026b). Compared with these tariffs, the benchmark's cap is stricter in level (3.5 instead of 8 kW), in time resolution (15 instead of 60 minutes) and in kind (a hard limit instead of a price). It therefore tests coordination under demanding conditions; replacing it with a priced peak is left for future work. The build procedure verifies with the MILP solver that every scenario has a feasible schedule; this held for all 48 scenarios without relaxing the limit. Table 5.2 reports the resulting characteristics per type. A scenario requires coordination when running every appliance at its individually cheapest start time exceeds the peak limit; these 28 scenarios form the basis of the conflict resolution rate.

Table 5.1 Evaluation scenario design.

Type	Price pattern	Appliances	Inst.	Research purpose
A	Stable, low volatility	2 appliances: dishwasher + washing machine	8	Baseline check: both systems should perform well
B	Stable, low volatility	3 appliances: dishwasher + washing machine + tumble dryer	8	Tests three-way conflict resolution
C	Moderate volatility	3 to 5 appliances mixed	8	Tests price-signal responsiveness
D	High volatility, winter peak	5 appliances, overlapping constraints	8	Most financially consequential condition
E	Price spike (Dec 2022 crisis)	3 appliances, ready 0–2 h before the daily price peak, 4–8 h slack	8	Extreme Finnish price event robustness
F	Negative spot price (surplus)	4 appliances, flexible windows	8	Tests use of low-cost windows on negative spot-price days

Table 5.2 Benchmark characteristics per scenario type.

Type	Price days	Mean appliances	Require coordination	Naive schedule within limit	MILP saving vs. naive
A	2022–2026	2.0	1/8	8/8	10.2 %
B	2023–2026	3.0	4/8	7/8	4.3 %
C	2023–2026	3.4	5/8	6/8	19.9 %
D	2022–2026 (Dec–Feb)	5.0	8/8	0/8	37.0 %
E	1–14 Dec 2022	3.0	4/8	3/8	14.4 %
F	2023–2025	4.0	6/8	5/8	7.8 %
All		3.4	28/48	29/48	

5.2 Value of 15-minute price information

Since 1 October 2025 the Nord Pool day-ahead market settles Finnish electricity in fifteen-minute periods. The main benchmark uses hourly prices, because its type E days date from the 2022 energy crisis. To examine the new market design, a second set of 48 scenarios (Q-A01 to Q-F08) was built with the same procedure from native fifteen-minute FI prices for 1 October 2025 to 25 September 2026 (358 complete days), obtained from the public price service of the Estonian transmission system operator, which republishes Nord Pool data (Elering AS, 2026). Its hourly averages match the hourly series used in Section 5.1. Because this period contains no crisis month, its type E days are drawn from the 16 days with the highest peak price (up to 65 c/kWh), and no day is used by more than one type. Prices genuinely vary within the hour: in 98 % of hours the four quarter-hour prices differ, with a median spread of 0.75 c/kWh.

For each Q scenario, two optimal schedules were computed with the MILP solver. The first is planned on the real fifteen-minute prices. The second is planned on hourly averages, which is all the price information a household had before the switch, and is then settled at the real fifteen-minute prices. Hourly prices make the four quarters of an hour tie, so ties are broken towards the earliest start. The cost difference between the two schedules is the saving that is only reachable with fifteen-minute price information. Table 5.3 reports the savings of both schedules relative to the naive schedule, the additional cost reduction from fifteen-minute information, and the share of the achievable saving that requires it; the "All" row weights scenarios by their cost.

Table 5.3 Optimal schedule cost with hourly vs. 15-minute price information (MILP, Q scenarios).

Type	Intra-hour spread (c/kWh)	Saving, hourly plan	Saving, 15-min plan	Additional cost reduction	Share of saving needing 15-min	Scenarios improved
A	0.41	4.1 %	4.2 %	0.1 %	2.4 %	5/8
B	0.33	3.2 %	3.4 %	0.3 %	6.4 %	8/8
C	1.37	7.1 %	8.6 %	1.7 %	16.4 %	8/8
D	4.96	27.6 %	28.6 %	1.4 %	3.5 %	8/8
E	4.91	35.4 %	37.0 %	2.1 %	2.8 %	6/8
F	0.21	4.6 %	4.7 %	0.1 %	1.7 %	4/8
All	2.03	23.7 %	24.7 %	1.2 %	3.7 %	39/48

Fifteen-minute price information lowers the optimal cost in 39 of the 48 scenarios, but the effect is modest: 1.2 % overall, or 3.7 % of the achievable saving. The remaining 96 % of the saving is available to a scheduler that plans on hourly prices, since most of the value lies in moving appliance cycles between hours rather than between quarters of an hour. The additional reduction grows with the intra-hour price spread (Spearman $\rho = 0.66$ across scenarios) and is largest on volatile days: 1.4 % in type D, 2.1 % in type E, and 10.2 % in the single largest case (Q-E03, 17 October 2025). Type C shows the largest relative share

(16.4 %), because its hourly price differences are small, so intra-hour variation forms a larger part of the opportunity. In absolute terms the gain is about one cent per household per day for the two to five cycles scheduled, and about three cents on type D days. The schedules nevertheless differ: 110 of the 163 appliance start times change between the two plans. These results indicate that the hourly main benchmark captures most of the scheduling problem, while any difference between the agents under fifteen-minute prices is expected to appear mainly in types C, D and E.

5.3 Demonstration results

Table 5.4 shows what each system recommends for one day, scenario C03 (28 September 2023). The household has four appliance cycles: a 135-minute tumble dryer cycle ready from 06:00, a second 45-minute dryer cycle ready from 08:45, a dishwasher ready from 08:00 and a washing machine ready from 11:45, each with a deadline in the evening, and a peak load limit of 3.5 kW. Starting every cycle as soon as it is ready would cost €0.78.

Table 5.4 Daily recommendations for scenario C03 (28 September 2023), run 1 with GPT-4o-mini.

System	Dryer 1	Dryer 2	Dish-washer	Washing machine	Saving vs. no scheduling
No scheduling	06:00	08:45	08:00	11:45	–
MILP optimum	06:00	16:15	15:00	15:30	€0.139
EMHASS, profiles	06:00	16:15	15:00	15:30	€0.139
Smart timer (own heuristic)	06:00	16:00	15:00	16:00	€0.139
Multi-agent pipeline	06:00	16:00	15:00	16:00	€0.139
Single agent	06:00	16:00	15:00	15:15	€0.137
Cheapest hours	06:00	15:00	08:00	14:00	€0.080
LLM without tools	06:00	08:45	08:00	11:45	€0.000

The pipeline, the single agent, the smart timer and EMHASS all move the dishwasher, the second dryer and the washing machine into the cheap afternoon hours and capture almost the full saving of €0.14. The cheapest-hours automation moves the second dryer and the washing machine but leaves the dishwasher at its ready time: the blueprint starts a cycle at the first slot whose price is at or below its threshold, and the price at 08:00 already qualifies, although cheaper hours follow later. The language models also explain their recommendation. The pipeline's Orchestrator wrote: "The first tumble dryer will start at 06:00, followed by the dishwasher at 15:00, and the second tumble dryer and washing machine will both run at 16:00 … staying within your peak load limit of 3.50 kW." The explanation of the LLM without tools illustrates the risk of an ungrounded message: it states that the schedule "was optimized to run the appliances during the lowest price slots", while its start times are exactly those of no scheduling and save nothing. The pipeline's summary is checked against the final peak-load test before it is written (Section 4.2), whereas nothing checks the claims of the LLM without tools.

5.3.1 Design iterations

The evaluated pipeline is the result of five build-and-evaluate cycles, in line with the iterative design and demonstration activities of DSR (Peffers et al., 2007). The first four used a pilot of six scenarios of the main benchmark, one per type (A01, B03, C02, D01, E01 and F04); the fifth used the full main benchmark. Because the pipeline was refined on these scenarios, the main benchmark is treated as exploratory, and the hypotheses were confirmed on the held-out benchmark (Section 5.7). Table 5.5 summarizes each iteration. Every change is a general mechanism that applies to all scenarios and, where it concerns errors of the language model, equally to the single agent; none was tuned to an individual scenario.

Table 5.5 Design iterations of the pipeline (pilot: six scenarios, one run each; iteration 5: 48 scenarios, three runs each).

	Problem observed	Change	Effect
1	The Resolver moved appliances back and forth until it ran out of steps, and the final summary could report success for a schedule that still exceeded the peak limit.	Automatic mode of `adjust_schedule`; the summary is written from the result of a final peak-load check.	Pipeline with GPT-4o-mini: valid schedules 3/6 to 6/6, mean gap 11.4 % to 0.6 %.
2	With the open-weight models, the provider rejected malformed tool calls, and the local model often ended its turn in prose without the required structured output.	Recovery nudge (at most two per agent and run) for both cases, applied identically to every agent and logged.	Single agent with gpt-oss-20b: valid schedules 4/6 to 6/6 once prose endings were also covered.
3	The Orchestrator's task brief sometimes omitted an appliance, which was then never scheduled.	The daily plan travels with the brief to the Profiler; the verification step checks that every appliance has a start and returns missing ones to the Scheduler.	Pipeline with GPT-4o-mini: mean gap 0.63 % to 0.36 %, with feedback rounds in some runs; no improvement with gpt-oss-20b.
4	With the step budget divided equally (25 steps per specialist), the Scheduler ran out of steps.	Budget divided by workload: 15 for the Profiler, 35 for the Scheduler and 25 for the Resolver, still 75 in total.	Scheduler step-limit failures 3 to 0 across the pilot; local pipeline valid schedules 1/6 to 2/6.
5	On the full benchmark, the only two invalid pipeline schedules started a tumble dryer cycle outside its time window.	The verification step also checks every start against its window and returns violations to the Scheduler.	Valid schedules 142/144 to 144/144, mean gap 1.50 % to 1.25 %, type E 2.30 % to 0.21 %.

Three observations follow from the iterations. First, most problems concerned reliability rather than cost: the agents usually found cheap start times, but a schedule had to be complete, within the peak limit and within every window before its cost counted. This is why the verification step grew from a peak-load check into a check of all three conditions. Second, the changes that helped GPT-4o-mini did little for the small local model. In every complete pilot from iteration 2 onwards its pipeline produced only one or two valid schedules out of six, while the single agent with the same model reached six out of six once malformed outputs were recovered, an early sign of the result for SQ2. Third, the verification step can trade cost for validity. Only seven of the 144 runs of iteration 5 needed a feedback round, and in one of them (C08, run 3) the repaired schedule was valid but 17.3 % above the optimum, against 0.4 and 0.5 % in the two runs that needed no repair. Over the whole benchmark the window check improved validity and the mean gap, but the change in gap is not statistically significant (Wilcoxon $p = 0.65$, $N = 48$).

5.4 Quantitative results

Work in progress.

5.5 Statistical analysis

This section reports the tests described in Section 3.3 for GPT-4o-mini on the main benchmark, with the mean over three runs per scenario as the paired observation ($N = 48$). These results are exploratory; the pre-registered confirmatory results are reported in Section 5.7. The results for gpt-oss-20b are added when its runs are complete.

Schedule quality. The multi-agent pipeline reached a mean optimality gap of 1.25 % and the single agent 1.43 %. The Wilcoxon signed-rank test finds no significant difference ($W = 449$, $Z = -0.14$, $p = 0.89$, $r = 0.02$), and the median paired difference is zero, because both designs reach the same cost in most scenarios. The equivalence test shows that this is not merely a lack of evidence. The mean paired difference is -0.18 percentage points, with a 90 % confidence interval of $[-0.71, 0.35]$, which lies inside the primary margin of ± 1 percentage point (Table 5.6). The two designs are therefore equivalent in schedule quality within one percentage point of the optimum. At the narrower margin of 0.5 percentage points the t version of the test does not conclude equivalence, because a few scenarios with large differences in either direction widen the interval, although the Wilcoxon version does.

Efficiency. The two designs differ significantly in every efficiency measure (Table 5.7). The pipeline used 34 % fewer input tokens, because each specialist works with a short context and only its own tools, while the single agent carries the entire conversation and all eight tool descriptions through every step. It produced 83 % more output tokens and

Table 5.6 Equivalence test (TOST) of the optimality gap, multi-agent vs. single-agent (GPT-4o-mini, $N = 48$).

Margin	p, paired t TOST	p, Wilcoxon TOST	Equivalent
± 0.5 percentage points	0.156	< 0.001	No
± 1 percentage point (primary)	0.006	< 0.001	Yes
± 2 percentage points	< 0.001	< 0.001	Yes

made 23 % more tool calls, since four agents each write their own structured output and call their tools separately. Input tokens dominate the price, so the API cost per run was 21 % lower for the pipeline, and lower in 35 of the 48 scenarios. The sequential hand-offs made the pipeline 69 % slower, with a mean of 21.4 seconds against 12.7 seconds, and it was faster in only two scenarios.

Table 5.7 Efficiency per run, multi-agent vs. single-agent (GPT-4o-mini; tokens and tool calls per run, API cost in USD, latency in seconds; each value is the mean over $N = 48$ scenarios of the per-scenario mean of three runs; Wilcoxon signed-rank test).

Measure	Single agent	Multi-agent	Difference	Multi-agent lower	p	r
Input tokens	21 686	14 292	-34 %	43/48	< 0.001	0.79
Output tokens	661	1 208	$+83$ %	1/48	< 0.001	0.87
Total tokens	22 347	15 500	-31 %	40/48	< 0.001	0.75
API cost (USD)	0.0037	0.0029	-21 %	35/48	< 0.001	0.54
Latency (s)	12.7	21.4	$+69$ %	2/48	< 0.001	0.86
Tool calls	16.5	20.3	$+23$ %	1/48	< 0.001	0.81

Taken together, decomposing the task into specialist agents did not lower the cost of the schedules, but it reached the same quality with about a third fewer tokens and a fifth lower inference cost, at the price of a longer response time.

5.6 Comparison with baselines and real tools

Table 5.8 compares the two agent designs with the smart timer, the LLM without tools and the two real home energy management tools. All systems that use a language model use GPT-4o-mini.

Without tools, GPT-4o-mini schedules poorly. Its mean optimality gap is 23.1 %, and 18 of its 144 schedules are infeasible: 14 exceed the peak limit and 4 start a cycle outside its window. A further 22 of its feasible schedules cost more than starting every appliance as soon as it is ready, and over all its runs it captures only 13 % of the achievable saving, against about 90 % for the agents. Both agent designs are significantly better (Wilcoxon $p < 0.001$, $r = 0.87$ for each), with median paired differences of 10.8 percentage points

Table 5.8 Agent designs, simpler baselines and real home energy management tools (GPT-4o-mini, main benchmark, 48 scenarios).

System	Feasible runs	Mean gap	Median gap	Gap, type D	Savings captured	Time per run
Smart timer (own heuristic)	48/48	0.23 %	0.00 %	1.2 %	99 %	$<$ 0.01 s
EMHASS, profiles	48/48	0.32 %	0.00 %	0.1 %	99 %	1.2 s
Multi-agent pipeline	144/144	1.25 %	0.01 %	5.9 %	93 %	21.4 s
Single agent	143/144	1.43 %	0.00 %	6.5 %	90 %	12.7 s
EMHASS, nominal power	48/48	2.24 %	0.45 %	8.5 %	87 %	1.0 s
Cheapest hours	25/48	17.6 %	4.2 %	62.7 %	25 %	$<$ 0.01 s
LLM without tools	126/144	23.1 %	11.9 %	66.0 %	13 %	2.0 s

for the single agent and 11.4 for the pipeline. The same model that fails to schedule from a single prompt comes close to the optimum when it can query prices, costs and loads through tools and check its own work. Tool use, rather than the language model's own calculation, is therefore what makes the agents viable.

The smart timer, however, outperforms both agent designs. It finds a feasible schedule in every scenario, is optimal in 30 of the 48, and its largest gap is 3.7 % (D04). Both agent designs have significantly larger gaps than the timer (single agent $p < 0.001$, $r = 0.63$; pipeline $p < 0.001$, $r = 0.68$). The difference is concentrated in type D, where the timer's mean gap is 1.2 % against 5.9 % for the pipeline and 6.5 % for the single agent. The median paired differences are small (0.002 percentage points for the single agent and 0.017 for the pipeline), because in most scenarios all three systems reach the optimum or come very close to it. The timer needs no language model, runs in about two milliseconds and has no inference cost. In this benchmark, where the power profile and window of every appliance are known exactly and the horizon is one day, placing the appliances one at a time is almost always enough. The agents come within about one percentage point of this rule but do not improve on it. What they add is the ability to start from a request written in natural language rather than from structured appliance data, and to explain the schedule they produce.

The two real home energy management tools, added after the pre-registration, bracket the agents. Configured with the measured power profiles, EMHASS comes close to the optimum: every schedule is feasible, 32 of the 48 are optimal, and the mean gap is 0.32 %. Its largest gap, 4.2 % in F02, comes from its default solver tolerance of 1 % (the setting `lp_solver_mip_rel_gap` = 0.01; EMHASS contributors, 2026), which applies to the whole electricity bill including the base load and can therefore amount to several percent of the appliance cost scored here. With an exact tolerance the same optimization comes within 0.2 % of the optimum in every scenario, which confirms that the MILP reference reflects what a deployed open-source optimizer can achieve. The agents' mean gaps are about one percentage point higher, but the difference from EMHASS as deployed is not significant (Wilcoxon $p = 0.24$ for the single agent and $p = 0.19$ for the pipeline). Configured with a constant nominal power, EMHASS still keeps every schedule within the peak limit, because it plans each cycle at its peak power, but it misjudges the cost

of the real cycles and reaches a mean gap of 2.2 %. Both agent designs are significantly better than this configuration (Wilcoxon $p < 0.001$ for each, $r = 0.67$ for the single agent and 0.64 for the pipeline). The cheapest-hours automation, under the assumption stated in Section 3.3 that its trigger starts a cycle which then runs to completion, exceeds the peak limit in 23 of the 48 scenarios, because each appliance follows its own cheap period and appliances whose windows share a price minimum start together. Scored at the naive cost when infeasible, it reaches a mean gap of 17.6 %, and both agent designs are significantly better ($p < 0.001$, $r = 0.82$ and 0.81). The agents are therefore outperformed by the smart timer and are not significantly different from a well-configured optimizer as deployed, while they outperform a simply configured optimizer and the per-device automation that is common among households.

Two checks show that this ranking does not depend on how the costs are scored. First, scoring an infeasible run at the cost of the naive schedule does not drive the result. Restricted to feasible runs, the mean gaps are 0.23 % for the smart timer, 0.32 % for EMHASS with profiles, 1.25 % for the pipeline, 1.45 % for the single agent, 2.24 % for EMHASS with nominal power, 7.9 % for the cheapest-hours automation and 23.0 % for the LLM without tools, the same order as in Table 5.8, and the order on the held-out benchmark is also unchanged. The cheapest-hours automation looks better on feasible runs only because these are the days on which its appliances happen not to collide. Second, the ranking is the same under the spot price alone as under the consumer price, as explained in Section 3.3. The Discussion returns to what these results imply for the artifact.

5.7 Confirmatory evaluation

The hypotheses of SQ1 and SQ2 were fixed in the pre-registration, which was committed to the project repository on 30 September 2026 together with the code, prompts, tools and analysis script. The confirmatory run followed on the held-out benchmark of 48 scenarios with fifteen-minute prices (Section 5.2), on which no language model had been run before. All systems that use a language model used GPT-4o-mini with three runs per scenario, and the analysis was run exactly as registered. The run cost USD 0.92 in API fees. Table 5.9 reports the six registered tests.

All six hypotheses are supported. For H1, both agent designs reached a mean optimality gap of about one percent (0.86 % for the single agent and 1.08 % for the pipeline), whereas the LLM without tools reached 18.8 %. Fifteen of its 144 schedules were infeasible, and over all its runs it captured only 24 % of the achievable saving, against 92 % for the single agent and 90 % for the pipeline. For H2a, the mean paired difference in gap was $+0.22$ percentage points with a 90 % confidence interval of $[-0.15, 0.59]$, which lies well inside the margin of ± 1 percentage point. On this benchmark the pipeline was thus marginally worse than the single agent on average, the reverse of the exploratory result, but in both runs the difference is small enough to count as equivalent. For H2b and H2c, the pipeline used 13 310 input tokens per run against 19 189 for the single agent, its API cost was 21 % lower, and its latency was 20.4 seconds against 13.4 seconds.

Table 5.9 Pre-registered confirmatory tests on the held-out fifteen-minute benchmark (GPT-4o-mini, $N = 48$; p-values Holm-adjusted within each sub-question).

	Hypothesis	Estimate	p	r	Supported
H1	Single agent gap < LLM without tools	-17.9 pp	< 0.001	0.86	Yes
H1	Pipeline gap < LLM without tools	-17.7 pp	< 0.001	0.86	Yes
H2a	Pipeline equivalent to single agent (± 1 pp)	$+0.22$ pp	< 0.001	–	Yes
H2b	Pipeline uses fewer input tokens	$-31\,\%$	< 0.001	0.84	Yes
H2b	Pipeline has a lower API cost per run	$-21\,\%$	< 0.001	0.70	Yes
H2c	Pipeline has a longer latency	$+52\,\%$	< 0.001	0.80	Yes

The confirmatory estimates are close to the exploratory ones: input tokens fell by 31 % in both runs, API cost by 21 % in both, and latency rose by 52 % here against 69 % before. The secondary results also replicate. The equivalence test is again not significant at the narrower margin of ± 0.5 percentage points in its t version ($p = 0.11$), although its Wilcoxon version is. The smart timer again outperforms both agent designs, with a mean gap of 0.17 % (Wilcoxon $p < 0.001$ against each design, $r = 0.69$ for the single agent and 0.64 for the pipeline). Type D remains the hardest scenario type, with mean gaps of 3.9 % for the single agent and 5.5 % for the pipeline, while the LLM without tools fails most clearly in type E, with a mean gap of 56 %.

The real home energy management tools were added after the pre-registration, so the following comparisons are secondary and not registered; their p-values are descriptive and not adjusted for multiple testing. EMHASS as deployed reaches a mean gap of 0.36 % with measured profiles, not significantly different from either agent design (Wilcoxon $p = 0.90$ for the single agent and $p = 0.69$ for the pipeline), and 3.5 % with nominal power, significantly worse than both ($p < 0.001$). The cheapest-hours automation exceeds the peak limit in 26 of the 48 scenarios, with a mean gap of 18.7 %. These patterns match those on the main benchmark.

The registered findings, H1 for SQ1 and H2 for SQ2, are therefore not specific to the benchmark on which the pipeline was developed: they hold on unseen days with a different price resolution. They cover GPT-4o-mini only. The part of SQ2 that concerns a small model run locally (H3) is tested on the main benchmark and reported separately.

5.8 Per-appliance breakdown

Work in progress.

6. Discussion

Work in progress. This chapter will interpret the results in relation to the research questions, connect findings to the agentic AI trends reviewed in Chapter 2, and address the limitations and directions for future research.

6.1 Interpretation of research questions

Work in progress.

6.2 Limitations

The study has five main limitations. First, the evaluation is a simulation on historical data. Real consumption profiles and real Nord Pool FI prices make it more realistic than synthetic scenarios, but the system was not tested in a live deployment, where appliance states change during the day and users may not keep to the windows they gave.

Second, the appliance data come from UK homes. The REFIT dataset was collected from 20 UK houses between 2013 and 2015, of which 18 are used, and may not reflect the appliances and habits of Finnish households today. No public Finnish dataset measures washing machines, dishwashers and tumble dryers separately over time. Fingrid's Datahub open data publish the hourly consumption of Finnish metering points only in aggregated form, by user group (Fingrid, 2024), and the Finnish household electricity use study reports measured annual consumption per appliance group from about one hundred homes, without load time series (Adato Energia, 2008). Appliance-level datasets from other countries, such as a Norwegian dataset of a single household over 24 days, are too small to replace REFIT. The cycle profiles should transfer, because the heating phase that dominates the energy and peak power of a wash cycle depends on the machine and not on the country. The base load transfers less well. Many Finnish homes have electric space heating, an electric sauna or a heat pump, whose power far exceeds that of the UK homes in REFIT, so in a real Finnish house the peak limit, in particular the realistic 8 kW limit, would bind more often than in these scenarios.

Third, LLM output is non-deterministic, and two runs of the same scenario may produce different schedules. Each scenario is run three times and the results are averaged, which reduces but does not remove this variance. Fourth, only three appliance types are studied; electric vehicle chargers, heat pumps and battery storage would need their own evaluation. Fifth, the daily appliance plan is given in structured form, with an explicit window for every appliance. The study therefore does not test whether the agents can interpret wishes stated informally, which is where they are expected to have an advantage over rule-based scheduling. On the first benchmark, where every problem was fully specified, the smart

timer, a simple rule written for this study, saved more than the agents (Section 5.6).

6.3 Future research

Several directions for future research emerge from this study. First, extending the pipeline to thermal loads such as electric space heating, heat pumps and domestic hot water systems would address the gap explicitly identified by El Makroum et al. (2025). Second, implementing the price API and data reader as MCP servers would enable reuse by future researchers without custom integration code, contributing to the emerging standardization of agent tool interfaces. Third, a live deployment study involving Finnish households on spot-price contracts would provide ecological validity that simulation-based evaluation cannot. Fourth, extending the multi-LLM comparison to include Qwen-3-32B and Phi-4 would provide a more complete picture of the SLM landscape for residential scheduling tasks. Fifth, a benchmark of informal household requests with annotated constraints would test the agents where a rule-based scheduler cannot operate, by measuring how accurately each system turns such requests into a correct schedule. Sixth, the hard peak cap of the benchmark could be replaced by the power charge of a real distribution tariff, so that a system weighs the energy saving of a cheap hour against the cost of raising the household's monthly peak, as under the Energy Authority's guidance for power-based distribution pricing.

7. Conclusions

Work in progress. This chapter will summarize the main contributions of the thesis, explicitly answer the RQ, SQ1 and SQ2 using evidence from Chapter 5, and state the implications for academic research on multi-agent LLM systems and practical deployment of intelligent demand flexibility tools in Finnish residential settings.

References

Aalto University. (2026, March 23). *Energy poverty is not recognized in Finland* [News release on a study by S. Numminen and S. Karhinen, Aalto University and Finnish Environment Institute]. https://www.aalto.fi/en/news/energy-poverty-is-not-recognized-in-finland

Adato Energia. (2008). *Kotitalouksien sähkönkäyttö 2006* [Household electricity use 2006]. Adato Energia Oy. ISBN 978-952-9696-41-3.

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM. https://doi.org/10.1145/3290605.3300233

Anthropic. (2024). *Model Context Protocol*. https://modelcontextprotocol.io

Beaudin, M., & Zareipour, H. (2015). Home energy management systems: A review of modelling and complexity. *Renewable and Sustainable Energy Reviews, 45*, 318–335. https://doi.org/10.1016/j.rser.2015.01.046

Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small language models are the future of agentic AI* [Position paper, NVIDIA Research]. arXiv:2506.02153. https://doi.org/10.48550/arXiv.2506.02153

Caruna. (2026, March 20). *The strategic stockpile fee included in the electricity tax will increase from 1 April 2026* [News release]. https://caruna.fi/en/news/strategic-stockpile-fee-included-electricity-tax-will-increase-1-april-2026

El Makroum, R., Zwickl-Bernhard, S., & Kranzl, L. (2025). Agentic AI home energy management system: A large language model framework for residential load scheduling [Preprint]. arXiv:2510.26603. https://doi.org/10.48550/arXiv.2510.26603

Elering AS. (2026). *Nord Pool day-ahead prices, bidding zone FI* [Data service; data owner Nord Pool]. Retrieved 26 September 2026, from https://dashboard.elering.ee/api/nps/price

dala318. (n.d.). *nordpool_planner* [Home Assistant custom component]. GitHub. Retrieved 5 October 2026, from https://github.com/dala318/nordpool_planner

EMHASS contributors. (2026). *EMHASS: Energy Management for Home Assistant* (Version 0.18.4) [Computer software]. https://emhass.readthedocs.io

Energy Authority (Energiavirasto). (2025, July 10). *Dynamic-price contracts held by more than one third of households: Fixed-term contracts still the most common type in 2024* [Press release]. https://energiavirasto.fi/en/-/dynamic-price-contracts-held-by-more-than-one-third-of-households-fixed-term-contracts-still-the-most-common-type-in-2024

ENTSO-E. (2026). *Transparency Platform: Day-ahead prices, bidding zone FI* [Data set]. European Network of Transmission System Operators for Electricity. Retrieved 6 October 2026, from https://transparency.entsoe.eu

Fingrid. (2024). *Fingrid's Open data service now provides data on electricity consumption and small-scale electricity production from accounting points in Finland* [News release]. Retrieved 5 October 2026, from https://www.fingrid.fi/en/news/news/2024/fingrids-open-data-service-now-provides-data-on-electricity-consumption-and-small-scale-electricity-production-from-accounting-points-in-finland/

Gokhale, G., Claessens, B., & Develder, C. (2024). Explainable home energy management systems based on reinforcement learning using differentiable decision trees. In *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems (e-Energy '24)* (pp. 687–691). ACM. https://doi.org/10.1145/3632775.3663310

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024)* (pp. 8048–8057).

Haakana, J., Haapaniemi, J., Lamminpää, A., Lassila, J., Viiliäinen, S., & Vähäsaari, M. (2025). Effects of extreme spot market prices on the electrical power system: Lessons from the Finnish distribution system. *IET Generation, Transmission & Distribution*, e70114. https://doi.org/10.1049/gtd2.70114

Han, S., Zhang, Q., Jin, W., & Xu, Z. (2024). *LLM multi-agent systems: Challenges and open problems* [Preprint]. arXiv:2402.03578. https://doi.org/10.48550/arXiv.2402.03578

Helen Sähköverkko. (2026a). *Network service products and prices*. Retrieved 4 October 2026, from https://www.helensahkoverkko.fi/en/private-customers/Information-about-electricity-consumption/Network-service-products-and-prices

Helen Sähköverkko. (2026b, February 16). *Energy Authority clarifies electricity distribution pricing: an option without a power charge remains for households* [News release]. https://www.helensahkoverkko.fi/en/uutiset/2026/ener

gy-authority-clarifies-electricity-distribution-pricing

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR 2024)*. https://doi.org/10.48550/arXiv.2308.00352

jisotalo. (n.d.). *shelly-porssisahko: Nordpool electricity spot price control for Shelly devices* (Version 3.4) [Computer software]. GitHub. Retrieved 5 October 2026, from https://github.com/jisotalo/shelly-porssisahko-en

Jonah, S., Moses, Q., Babatunde, A., Ajao-Olarinoye, M., & Bammeke, D. (2026). LLMs for agentic home energy management [Preprint]. arXiv:2607.04569. https://doi.org/10.48550/arXiv.2607.04569

Kert, K. (n.d.). *Nordpool cheapest hours actions* [Home Assistant blueprint]. GitHub. Retrieved 3 October 2026, from https://github.com/kullarkert/ha_nordpool_hours_blueprint

Lakens, D. (2017). Equivalence tests: A practical primer for *t* tests, correlations, and meta-analyses. *Social Psychological and Personality Science, 8*(4), 355–362. https://doi.org/10.1177/1948550617697177

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 9459–9474).

Li, B., Mellou, K., Zhang, B., Pathuri, J., & Menache, I. (2023). *Large language models for supply chain optimization* [Preprint]. arXiv:2307.03875. https://doi.org/10.48550/arXiv.2307.03875

Liu, Y., Lo, S. K., Lu, Q., Zhu, L., Zhao, D., Xu, X., Harrer, S., & Whittle, J. (2025). Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents. *Journal of Systems and Software, 220*, 112278. https://doi.org/10.1016/j.jss.2024.112278

Luccioni, S., Jernite, Y., & Strubell, E. (2024). Power hungry processing: Watts driving the cost of AI deployment? In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)* (pp. 85–99). https://doi.org/10.1145/3630106.3658542

March, S. T., & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems, 15*(4), 251–266.

Mocanu, E., Mocanu, D. C., Nguyen, P. H., Liotta, A., Webber, M. E., Gibescu, M., & Slootweg, J. G. (2019). On-line building energy optimization using deep reinforcement learning. *IEEE Transactions on Smart Grid, 10*(4), 3698–3708. https://doi.org/10.1109/TSG.2018.2834219

Moradzadeh, B., & Tomsovic, K. (2013). Two-stage residential energy management considering network operational constraints. *IEEE Transactions on Smart Grid, 4*(4), 2339–2346. https://doi.org/10.1109/TSG.2013.2265313

Murray, D., & Stankovic, L. (2016). *REFIT: Electrical load measurements (cleaned)* [Data set]. University of Strathclyde. https://doi.org/10.15129/9ab14b0e-19ac-4279-938f-27f643078cec

Murray, D., Stankovic, L., & Stankovic, V. (2017). An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study. *Scientific Data, 4*, 160122. https://doi.org/10.1038/sdata.2016.122

OpenHEMS contributors. (2026). *OpenHEMS: Home energy management system based on Home Assistant* [Computer software]. GitHub. Retrieved 5 October 2026, from https://github.com/abriotde/openhems-sample

Pakastin Oy. (2026). *Sähkötin: Nord Pool FI day-ahead spot prices* [Data service]. Retrieved 25 September 2026, from https://sahkotin.fi

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77.

Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics, 15*(6), 657–680. https://doi.org/10.1007/BF01068419

Sou, K. C., Weimer, J., Sandberg, H., & Johansson, K. H. (2011). Scheduling smart home appliances using mixed integer linear programming. In *Proceedings of the 50th IEEE Conference on Decision and Control and European Control Conference* (pp. 5144–5149). IEEE. https://doi.org/10.1109/CDC.2011.6161081

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*, 186345. https://doi.org/10.1007/s11704-024-40231-1

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.03629.

Zharova, A., Boer, A., Knoblauch, J., Schewina, K. I., & Vihs, J. (2024). An explainable multi-agent recommendation system for energy-efficient decision support in smart

homes. *Environmental Data Science, 3*, e7. https://doi.org/10.1017/eds.2024.8